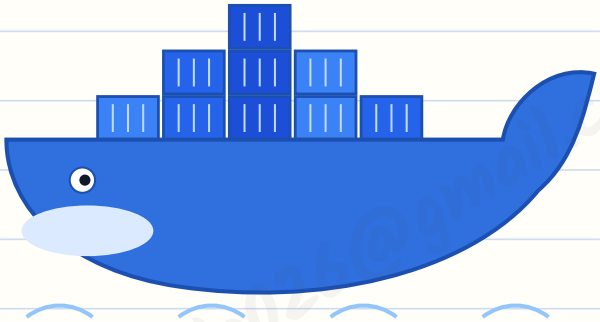
Part 2 of 2

# Docker

## Build, Run & Ship

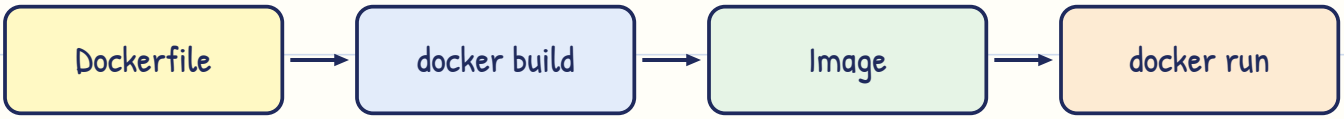
Dockerfile to production + memory map + interview Q&A



### What's inside


2. Dockerfile basics
3. CMD, ENTRYPOINT & friends
4. Docker networking
5. Docker storage
6. Docker Compose
7. Multi-stage builds & security
8. Limits & troubleshooting
9. Memory map of Docker
10. Interview Q&A


### From code to running app




**Before you start**

New here? Read ***Part 1*** first (what Docker is, images, containers, commands). Then build every example in this part.

# Docker Notes

## Dockerfile Basics

A Dockerfile is a text file of instructions that Docker reads top to bottom to build an image.

### Sample: a Python Flask app

```
FROM python:3.12-slim              # base image
WORKDIR /app                       # working folder
COPY requirements.txt .            # deps list first
RUN pip install -r requirements.txt  # install
COPY . .                           # then your code
ENV FLASK_ENV=production           # variable
EXPOSE 5000                        # document port
CMD ["python", "app.py"]           # default command
```

| Instruction | What it does |
|---|---|
| FROM | Base image to start from |
| WORKDIR | Sets the working directory |
| COPY | Copies files from build context into image |
| RUN | Runs a command at build time |
| ENV | Sets an environment variable (no real secrets!) |
| EXPOSE | Documents the listening port |
| CMD | Default command when container starts |
| ENTRYPOINT | Main executable of the container |
| USER | Run as this user (prefer non-root) |
| HEALTHCHECK | Test whether the app is healthy |

Order matters: files that change least go first, so Docker can ***reuse cached layers*** and rebuild faster.

ashishdawande026@gmail.com

# Docker Notes

## CMD, ENTRYPOINT & Friends

| | CMD | ENTRYPOINT |
|---|---|---|
| Role | Default command or arguments | The main executable |
| Overridden? | Easily: docker run img other | Only with --entrypoint |
| Typical use | Sensible defaults | Container always runs one program |

```
ENTRYPOINT ["python"]
CMD ["app.py"]          # docker run img test.py  >  python test.py
```

| Pair | First | Second |
|---|---|---|
| COPY vs ADD | COPY: plain file copy (preferred) | ADD: also URLs and auto-extracts tar |
| ARG vs ENV | ARG: build time only | ENV: also inside running container |
| EXPOSE vs -p | EXPOSE: only documents a port | -p: really publishes it to host |
| Exec vs shell form | CMD ["a","b"]: recommended, gets signals | CMD a b: runs via a shell |

### .dockerignore & build context

The ***build context*** is every file sent to Docker when you build. A .dockerignore keeps junk and secrets out.

```
.git
node_modules
.env
*.log
```

**Why bother?**

Smaller context = faster builds and no accidental secrets inside your image.

**Exam Tip:**

Exam hint: ***CMD can be overridden, ENTRYPOINT is hard to override.*** Use both: ENTRYPOINT = program, CMD = default arguments.

# Docker Notes

## Docker Networking

| Driver | What it does | Use when |
|---|---|---|
| bridge | Private network on one host (default) | Most single-host apps |
| host | Shares the host network, no isolation | Max network speed (Linux) |
| none | No network at all | Full isolation |
| overlay | Connects containers across hosts | Swarm / multi-host |
| macvlan | Container gets its own LAN address | Legacy apps needing a LAN IP |

### User-defined bridge: containers find each other by name




```
$ docker network create app-net
$ docker run -d --name db --network app-net mysql:8
$ docker run -d --name web --network app-net -p 8080:80 myapp
```

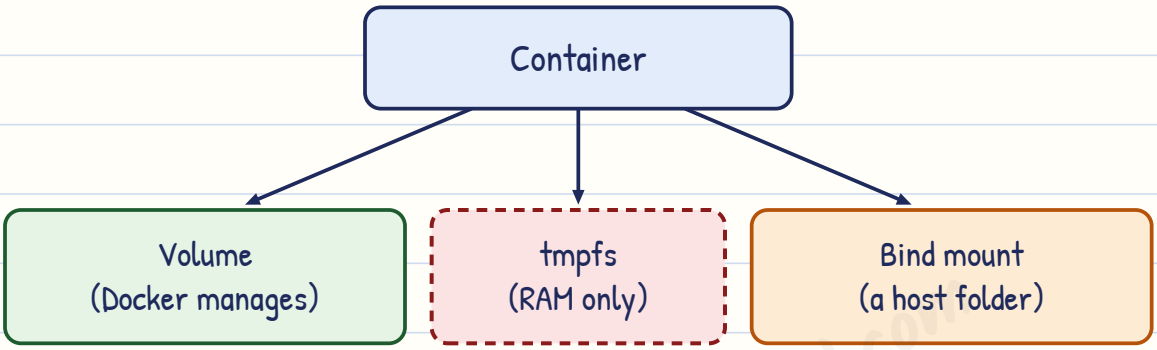
**Exam Tip:**
Default bridge has ***no name lookup***. A user-defined bridge has built-in DNS.

**Trick**
Containers on the same network talk ***without*** -p. Publish only what the outside world needs.

# Docker Notes

## Docker Storage

A container's writable layer is temporary. Remove the container and that data is gone. For data that must survive, use a ***volume*** or ***bind mount***.




| *Type* | *Managed by* | *Best for* |
|---|---|---|
| Named volume | Docker | Databases, app data (production) |
| Bind mount | Host filesystem | Development, live code, config files |
| tmpfs | Memory (RAM) | Temporary or sensitive data |
| Writable layer | Container | Throwaway changes only |

### Commands

```
$ docker volume create db-data
$ docker run -d -v db-data:/var/lib/mysql mysql:8   # named volume
$ docker run -v $(pwd):/app node:20                 # bind mount
$ docker volume ls   |   docker volume inspect db-data
```

**Exam Tip:**

docker rm does ***not*** delete named volumes.
docker volume prune removes unused ones.

**Memory Hook**

Volumes for ***production***, bind mounts for ***development*** (edit on host, see changes instantly).

ashishdawande026@gmail.com

# Docker Notes

## Docker Compose

Compose defines and runs multi-container apps from one YAML file.

```
services:
  app:
    build: .
    ports: ["8080:5000"]
    environment:
      DB_HOST: db
    depends_on:
      db:
        condition: service_healthy
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASS}
    volumes: [db-data:/var/lib/mysql]
    healthcheck:
      test: ["CMD","mysqladmin","ping"]
volumes:
  db-data:
```

### Read it like this

- Two ***services***: app and db
- db is the hostname the app uses, ***not*** localhost
- Data lives in the ***named volume***
- Password comes from a .env file
- App waits until db is ***healthy***

### Useful commands

`docker compose up -d`
Build if needed + start

`docker compose down`
Stop and remove containers

`docker compose ps`
List services

`docker compose logs -f`
Follow logs

`docker compose build`
Rebuild images

`docker compose exec app sh`
Shell in a service

**Careful**

down keeps volumes. Add -v to delete them too (your data!).

**Exam Tip:**

depends_on = start order. With a ***healthcheck*** condition it waits until the app is really ready.

ashishdawande026@gmail.com

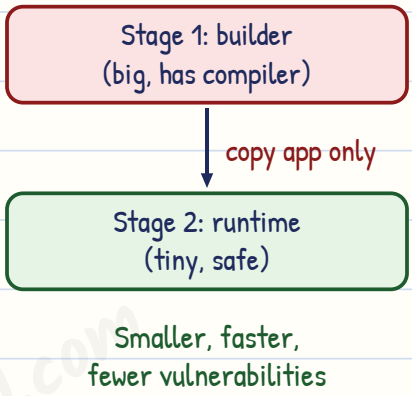
# Docker Notes

## Multi-stage Builds & Security

***Multi-stage build***: build in one stage, run in a tiny final stage. Compilers and source never reach production.

```
FROM golang:1.24 AS builder
WORKDIR /src
COPY . .
RUN go build -o app .

FROM gcr.io/distroless/base-debian12
COPY --from=builder /src/app /app
EXPOSE 8080
CMD ["/app"]
```




| *Security control* | *How* |
|---|---|
| Run as non-root | USER appuser and fix file ownership |
| No secrets in images | Inject at runtime (env, secrets), never ENV/COPY |
| Drop privileges | --cap-drop ALL, --read-only, never --privileged |
| Scan images | Scan in CI (e.g. Docker Scout, Trivy) |
| Minimal base image | slim / alpine / distroless, rebuild for patches |
| Least exposure | Publish only needed ports; do not mount docker.sock |

**Golden Rule**

Anything in an image layer can be read later, even if you delete it in the next layer. Never bake in passwords or API keys.

ashishdawande026@gmail.com

# Docker Notes

## Limits & Troubleshooting

### Container stops immediately? Check in this order

docker ps -a → docker logs → docker inspect → Exit code → CMD / ENTRYPOINT

### Exit codes

| 0 | 1 | 137 | 143 |
|---|---|---|---|
| Finished OK | App error | SIGKILL / OOM | SIGTERM (stop) |

### Resource limits

```
$ docker run -d --memory 512m --cpus 1.0 myapp
$ docker stats          # live CPU / memory per container
```

| Problem | What to check |
|---|---|
| Image pull fails | Name / tag, registry login, network, CPU architecture |
| Works locally, fails in prod | Image digest, env vars, mounts, permissions, limits |
| Keeps restarting | Logs, exit code, restart policy, healthcheck, memory |
| App unreachable | -p mapping, listens on 0.0.0.0 (not localhost), firewall |
| Out of memory (OOM) | docker stats, memory limit, leak? exit code 137 |
| Disk full | docker system df, then prune what is safe |
| Data disappeared | It was in the writable layer. Use a volume next time |

**Golden Rule**

Always start with docker logs and docker inspect. They answer most problems.

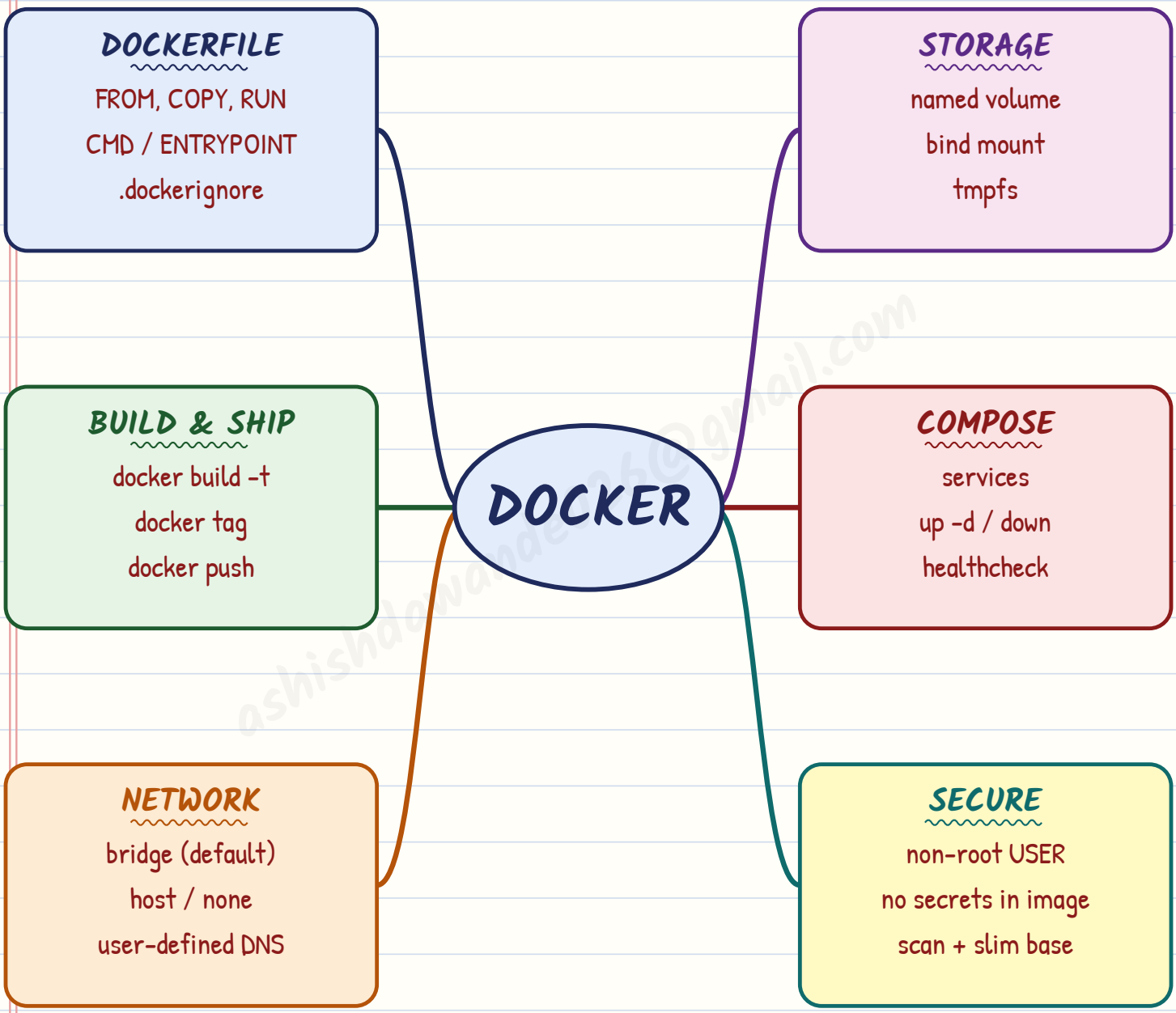
**Trick**

Reproduce it: docker run -it --entrypoint sh IMAGE opens a shell in the same image.

ashishdawande026@gmail.com

# Docker Notes

## Memory Map: Build, Run & Ship




ashishdawande026@gmail.com

**Memory Hook**

**Dockerfile order:** FROM → WORKDIR → COPY → RUN → EXPOSE → CMD. "Fast Whales Carry Rocket Engines Cleanly"

# Docker Notes

## Interview Q&A

**Q1** Image vs container?

Image = immutable template. Container = running instance with a writable layer.

**Q2** Docker vs VM?

Containers share the host kernel; a VM runs its own guest OS and kernel.

**Q3** CMD vs ENTRYPOINT?

CMD gives defaults and is easy to override. ENTRYPOINT is the main executable.

**Q4** EXPOSE vs -p?

EXPOSE only documents a port. -p publishes it on the host.

**Q5** Volume vs bind mount?

Volume is managed by Docker. Bind mount maps a specific host path.

**Q6** stop vs kill?

stop is graceful (SIGTERM first). kill is immediate (SIGKILL).

**Q7** run vs exec?

run creates a new container. exec runs a command in a running one.

**Q8** Compose vs Dockerfile?

Dockerfile builds one image. Compose runs multi-container apps.

**Q9** Tag vs digest?

A tag can move. A digest identifies exact image content.

**Interview Tip:**

Explain with a real example: "my Flask app + MySQL ran with Compose".

**Security**

Never put passwords or API keys inside an image or Dockerfile.

**Practice Daily . Build Once . Run Anywhere**

Save this post and share it with a friend learning Docker!

ashishdawande026@gmail.com